INF712-Computerunterschtützte Statistik 26-27WS

Vorlesung 01-02: Zufallszahlen und Simulation

Zufallszahlengenerierung

Computer sind deterministische Maschinen und können daher keine "echte" Zufälligkeit durch reine Berechnung erzeugen — stattdessen verwenden sie **Pseudozufallszahlengeneratoren (PRNGs)**. Ein PRNG beginnt mit einem Startwert, dem sogenannten **Seed**, und wendet darauf wiederholt eine deterministische mathematische Formel an, um eine Zahlenfolge zu erzeugen, die statistisch zufällig *erscheint* (gleichverteilt, unkorreliert, schwer vorhersagbar), obwohl sie bei bekanntem Seed und Algorithmus vollständig reproduzierbar ist. Genau diese Reproduzierbarkeit ist in der Statistik nützlich, da sie es erlaubt, Simulationen exakt zu wiederholen. Ein bekanntes Beispiel ist der **Mersenne-Twister (MT19937)**, der Standardgenerator vieler Statistik- und Programmiersprachen. Er besitzt eine extrem lange Periode von $2^{19937-1}$, arbeitet mit einem internen Zustand aus 624 Zahlen, der durch bitweise Operationen ("Twist") ständig neu erzeugt und vor der Ausgabe zusätzlich "temperiert" wird, um die statistische Qualität zu verbessern.

Für kryptografische Anwendungen reicht der Mersenne-Twister jedoch nicht aus, da sein interner Zustand aus nur 624 aufeinanderfolgenden Ausgabewerten rekonstruiert werden kann — ein Angreifer könnte damit alle zukünftigen Zahlen vorhersagen. Kryptografisch sichere PRNGs (**CSPRNGs**) lösen dieses Problem, indem sie auf Einwegfunktionen wie kryptografische Hashfunktionen basieren, die praktisch nicht umkehrbar sind, sodass selbst bei Kenntnis vieler Ausgaben der interne Zustand nicht rekonstruierbar ist. Wichtig ist jedoch: Auch CSPRNGs sind weiterhin **pseudozufällig**, nicht echt zufällig — sie bleiben deterministische Algorithmen. Echte Zufallszahlengeneratoren (**TRNGs**) hingegen nutzen physikalische Prozesse wie thermisches Rauschen, radioaktiven Zerfall oder Quanteneffekte, die grundsätzlich nicht-deterministisch sind. In der Praxis kombiniert man meist beide Ansätze: Ein TRNG liefert echte Entropie, um einen schnellen CSPRNG zu initialisieren und regelmäßig neu zu "seeden".

Die Wahl des Seeds hängt schließlich vom Anwendungszweck ab. Für statistische Simulationen genügt oft die Systemzeit als Seed, oder man setzt ihn manuell (z. B. `set.seed(42)`), um

Ergebnisse reproduzierbar zu machen. Für kryptografische Zwecke ist ein vorhersagbarer Seed jedoch ein Sicherheitsrisiko — hier sammelt das Betriebssystem Entropie aus unvorhersehbaren Quellen wie Tastatur- und Mausbewegungen, Festplattenzugriffszeiten oder Hardware-Rauschen, um den Seed und die laufende Neu-Initialisierung des CSPRNG abzusichern. Der zentrale Unterschied liegt also darin, ob Vorhersagbarkeit für die jeweilige Anwendung tolerierbar ist: In der Statistik ist sie unproblematisch oder sogar erwünscht, in der Kryptografie hingegen sicherheitskritisch.